
SL ASSIGNMENT 02

PRN : 260240128031

Name : *Pratiksha Dhembare*

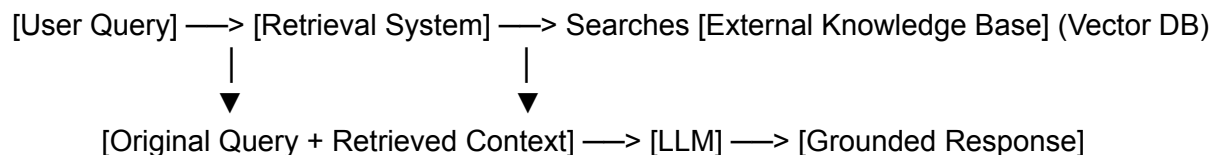
PRN : 260240128053

Name : *Vijay Rajage*

AI Trends & Practical Knowledge

1. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an architectural pattern designed to optimize the output of a Large Language Model (LLM) by referencing an authoritative, external knowledge base outside of its training data before generating a response.



How it Combines LLMs with External Knowledge Sources:

1. **Ingestion & Vectorization:** External documents (PDFs, enterprise databases, wikis) are broken into smaller text segments (chunks). These chunks are passed through an embedding model to convert them into high-dimensional numerical vectors representing semantic meaning, which are stored in a **Vector Database**.
2. **Retrieval Phase:** When a user submits a query, the system converts the query into a vector using the same embedding model. It performs a similarity search (like Cosine Similarity) against the vector database to retrieve the top-\$k\$ most contextually relevant chunks.
3. **Augmentation Phase:** The retrieved text chunks are combined with the user's original query into a structured prompt payload (e.g., "*Answer the question using only the following context: [Retrieved Data]*").
4. **Generation Phase:** The LLM reads the augmented prompt and synthesizes a natural language response. Because the model is provided with the exact facts required, it relies on its linguistic reasoning capabilities rather than memorized data, drastically increasing accuracy.

2. The Latest Horizons in Artificial Intelligence

SL ASSIGNMENT 02

The AI landscape is shifting from static, text-based completion systems toward autonomous, multi-modal systems operating seamlessly across different environments.

- **Multimodal AI:** Early GenAI models were restricted to single modalities (primarily text). Modern architectures natively process and interweave disparate data streams—**text, audio, images, and video**—simultaneously. This allows for unified semantic understanding, enabling tasks like generating real-time audio commentary from a live video feed or diagnosing anomalies across codebases and visual schematics combined.
- **AI Agents & Autonomous Workflows:** Moving beyond passive "chatbots," AI agents can reason dynamically, split complex goals into sub-tasks, execute multi-step workflows, and utilize external tools via API calls (e.g., executing Python code, querying web search engines, or updating CRM software) with minimal human intervention.
- **Open-Source & Locally Deployable LLMs:** High-performance open-weight architectures (such as Meta's LLaMA series, Mistral, and Mixtral) have closed the capability gap with closed-source proprietary APIs. This democratizes AI, enabling enterprises to fine-tune, secure, and deploy powerful state-of-the-art models entirely on-premise or within private clouds to ensure absolute data sovereignty.
- **Edge AI & Model Compression:** Running heavy workloads on centralized cloud servers introduces latency, bandwidth constraints, and privacy issues. Techniques like **Quantization** (reducing weight precision from FP32 to INT8/INT4), **Pruning**, and **Knowledge Distillation** allow complex models to run locally on hardware endpoints like smartphones, laptops, and automotive chips.
- **Responsible AI & Guardrails:** As commercial deployment scales, engineering focus has pivoted heavily to safety. Implementations of real-time input/output verification layers (guardrails), automated bias mitigation frameworks, and robust copyright compliance filters ensure models remain mathematically constrained within ethical, legal, and operational boundaries.

Generative AI (GenAI) Deep-Dive

1. Large Language Models (LLMs): Architecture & Mechanism

A **Large Language Model (LLM)** is a deep learning system containing billions of parameters trained on massive textual datasets. At its fundamental core, an LLM operates as a highly advanced statistical autocomplete system: it computes the probability distribution of the next token (word or word fragment) given a preceding sequence of tokens.

The Transformer Architecture & Core Components:

Introduced in the seminal 2017 paper "*Attention Is All You Need*", the Transformer architecture completely replaced recurrent architectures (like LSTMs) by processing text sequences in parallel rather than sequentially.

SL ASSIGNMENT 02

Input Tokens → Embedding + Positional Encoding → Multi-Head Self-Attention → Feed-Forward Network → Next Token Probabilities

- **The Attention Mechanism (Self-Attention):** This mathematical mechanism allows the model to dynamically evaluate the relationship between *all* words in a sequence simultaneously, regardless of their linear distance from one another. For instance, in the sentence *"The bank of the river overflowed its boundaries because of the heavy rain, causing the local bank to close its doors,"* self-attention allows the model to map the first instance of "bank" to "river" and the second instance to "doors," dynamically resolving semantic ambiguity.
- **Vectors (Queries, Keys, Values):** The input tokens are converted into dense vector representations. For every token, the attention layer computes three vectors: a **Query (\$Q\$)**, a **Key (\$K\$)**, and a **Value (\$V\$)**. The attention score between tokens is derived by taking the dot product of the Query vector of one token with the Key vector of another, scaling it, and applying a Softmax function:
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$
- **Pre-training on Massive Datasets:** The training happens in two core phases. First, **Self-Supervised Pre-training** exposes the model to petabytes of diverse internet data, textbooks, and code. The model repeatedly guesses masked words, adjusting its internal weights via backpropagation. Second, **Alignment** via Reinforcement Learning from Human Feedback (RLHF) or Direct Preference Optimization (DPO) shapes the raw prediction engine into an instruction-following assistant.

2. Prompt Engineering: Frameworks & Structural Impact

Prompt engineering is the deterministic science of constructing input formulations that mathematically steer an LLM's latent space toward producing highly optimized, accurate, and structured output patterns.

Demonstration of Input Structure vs. Output Quality:

- **Suboptimal "Naive" Prompt:** > *"Tell me about photosynthesis."*
 - **Resulting Vulnerability:** The model receives zero guardrails regarding length, target audience, structural formatting, or stylistic constraints. It will output a generic, encyclopedic wall of text that may be overly technical or unsatisfyingly brief.
 - **Engineered "System-Driven" Prompt:**
 - "Role:** You are an expert biology educator specializing in clear, metaphorical communication.
 - Task:** Explain the molecular process of photosynthesis.
 - Audience:** A 12-year-old middle school student.
 - Constraints:** Use a simple kitchen/cooking analogy. Do not exceed 150 words. Avoid dense jargon (e.g., skip 'photophosphorylation').
-

SL ASSIGNMENT 02

Format: Present the explanation in exactly three sequential bullet points: 1) The Ingredients, 2) The Cooking Process, and 3) The Final Dish."

- **Resulting Quality:** The model's attention is constrained strictly to the required constraints. It filters out high-level jargon, formats the text exactly as requested for easy reading, and outputs a clear, high-signal response.

3. Fine-Tuning vs. Prompt Tuning

While both techniques adapt a model to perform specialized enterprise tasks, they operate on completely different layers of the machine learning stack.

Metric / Dimension	Fine-Tuning (Full or Parameter-Efficient)	Prompt Tuning (Soft Prompting)
Core Mechanism	Physically modifies and updates the internal neural weights of the base model.	Base model weights remain 100% frozen . Virtual embedding vectors are prepended to the prompt.
Computational Resource Cost	Extremely High. Requires significant GPU clusters, long training times, and deep backpropagation cycles.	Extremely Low. Requires minimal compute since gradients are only calculated for a few virtual tokens.
Storage & Infrastructure	Must save and deploy an entirely new copy of the model parameters for every separate task.	Keeps one base model instance; switches tasks instantly by loading a tiny file containing the virtual tokens.
Data Requirements	Requires thousands to millions of high-quality, task-specific labeled data rows.	Requires only a small, representative set of input-output examples.

SL ASSIGNMENT 02

4. Hallucinations in Generative AI: Root Causes & Mitigation

A **hallucination** occurs when an LLM generates a response that is syntactically coherent and grammatically perfect, but factually false, ungrounded, or completely disconnected from reality.

Why Do Hallucinations Occur?

- **Statistical Autocomplete Nature:** Models do not understand "truth"; they calculate token probabilities. If a factual gap exists in their latent space, they will seamlessly output the next most statistically probable word sequence, even if it is fiction.
- **Data Contamination & Conflicting Information:** Training sets often contain conflicting internet facts, satire, biases, or outdated information.
- **Over-generalization:** The model tries to match patterns too strongly, leading it to hallucinate connections that don't exist in reality.

Technical Mitigation Strategies:

1. **Implementation of Grounding Systems (RAG):** Forcing the model to dynamically look up documents before speaking narrows its search space. By supplying a strict rule like *"If the answer cannot be found in the attached documentation, explicitly output 'Information not found'"*, the model is structurally blocked from guessing.
 2. **Temperature and Top-P Parameter Control:** Setting the generation **temperature** closer to **0.0** makes the token selection deterministic, forcing the model to choose only the absolute highest-probability tokens rather than creative alternatives.
 3. **Few-Shot Prompting and Chain-of-Thought (CoT):** Providing structural input-output examples teaches the model the exact boundary of what is acceptable. Forcing the model to write out its step-by-step reasoning logic before presenting its final conclusion helps it cross-reference its own logic path, reducing structural processing errors.
-